

Handover — Moving Build Work to Claude Code

2026-08-01 · Companion to the Operating Manual v1.2 · Read S2 before starting a session

1. Where things run

Your reading is correct. This chat interface executes in a sandboxed container on Anthropic’s servers with no access to your disk. Everything produced so far was assembled there and handed to you as downloads. Nothing from your machine has been read, and no case material has been transmitted.

Tool	Runs where	Suitable for
Chat (this)	Anthropic container	Design, review, drafting documents. No disk access
Claude Code	Your machine; inference remote	Recommended. Installing, testing, debugging scripts — with fixtures only
Claude Cowork	Your machine; inference remote	Same data exposure, no advantage for CLI-heavy work
Local model only	Fully on premises	Anything touching case material

Claude Code is the right tool for the remaining build work: it can install packages, run the scripts, read tracebacks, and iterate. But the recommendation comes with one condition that is not optional, and it is the whole of Section 2.

2. The boundary

The constraint. Claude Code runs locally but sends context to Anthropic’s servers for inference. Any document content that enters a session leaves your premises. That is precisely what this project exists to prevent.

This is not hypothetical or edge-case. Content enters a session through many ordinary routes: the Read tool, Grep, a ``cat`` in a bash call, ``pdftotext`` output, a script that echoes what it is processing, a log file quoted while diagnosing a failure, or an error message pasted in by you that happens to contain a source sentence.

So the line is drawn by directory, and it is drawn generously:

Path	Claude Code	Why
<code>~/Claude_Stuff/cli_projects/</code>	Yes	All Claude Code project work lives here
<code>.../translation-tools/</code>	Yes	The kit: scripts and config. No case data ever
<code>.../translation-tools/fixtures/</code>	Yes	Synthetic, invented content
<code>~/translation-work/docs/</code>	Yes	These reference documents. No case content
<code>.../confidential-projects/**</code>	No	The entire container tree — all matters
<code>.../<project>/source/, translated/</code>	No	Originals and their translations
<code>.../<project>/work/, logs/</code>	No	OCR text layers, memory, and logs that

Path	Claude Code	Why
		quote segments
.../work/inventory/manifest.tsv, by-lang/	No	Lists of file paths; a filename carries party names, dates, case numbers
.../work/inventory/summary.txt	Counts only	No paths. This is the file that goes to the client
.../_shared/glossary/	No	Inside the denied tree. Reaching it needs the container mounted, which the guard already forbids during a session

2.1 Controls, in order of how much to trust them

First, structural separation. Always start a session from the kit directory, never from your home directory:

```
cd "$KIT" && claude      # correct
cd ~ && claude           # WRONG - ~/translation-work in scope
```

Second, and stronger: keep the case material unmounted while a session is running. The container mounts at ~/translation-work/confidential-projects; open it only for batch runs in a plain terminal and close it before starting Claude Code. A path that is not present cannot be read by any mechanism.

Third, permission rules as a backstop. The kit ships .claude/settings.json with deny rules covering the data paths. Adjust the username in the paths to match your account.

Do not rely on the deny rules alone. They are a useful second layer, but two caveats apply. Deny rules govern the built-in tools and cat-style commands; they do not reliably cover indirect reads through a script. And there are open reports against Claude Code of deny rules not being enforced in some cases. Treat the configuration as a seatbelt, not as the reason it is safe to proceed. The directory separation is the actual control.

2.2 If a task seems to need real data

It almost never does. The fixture generator produces synthetic Slovene legal documents that exercise every hard case in the pipeline — abbreviations that must not split sentences, case numbers and dates that must pass through untranslated, diacritics, and a 7,000-row table with a realistic repeat structure. It also writes a nested, two-drop, mixed-language tree under fixtures/drop/ — Slovene beside Croatian, English and Armenian, plus a file too short to classify — which is what triage is developed against. Development and debugging should use these exclusively.

```
cd "$KIT"
tr-fixtures fixtures/

# exercises the same code paths as the real corpus
tr-xlsx fixtures/dokazi-velika.xlsx --survey
```

If a genuine document is needed to reproduce a defect, describe the structure rather than sharing the content: "a paragraph where a case number is followed by an abbreviation and a four-digit year" is enough to build a fixture that reproduces it. If that fails, hand-redact an excerpt yourself before pasting.

3. Getting started in Claude Code

```
export KIT=~/.Claude_Stuff/cli_projects/translation-tools
mkdir -p "$KIT"
unzip translation-tools.zip -d "$KIT"
chmod +x "$KIT"/bin/*

# adjust the username in the deny paths to match your account
nano "$KIT"/.claude/settings.json

cd "$KIT"
claude
```

The kit contains a CLAUDE.md that states the boundary and the design invariants. Claude Code reads it at session start, so the constraint is in context before the first instruction. Read it yourself once as well — it is the shortest statement of what must not change and why.

The kit is already a git repository, so there is nothing to initialise. One setting is not optional after unpacking a fresh copy: the hooks that keep client material out of the repository do not run until it is told where they live, because a global core.hooksPath shadows .git/hooks entirely.

```
cd "$KIT" && git config core.hooksPath .githooks
```

tr-setup does this for you whenever the kit is already a repository. Either way, stage selectively — git add <path> — rather than git add -A, so a document copied in to reproduce a defect cannot ride along into a commit.

4. Tasks for the Claude Code session

In order. Each has an acceptance criterion so completion is unambiguous.

#	Task	Done when
1	Run tr-setup; resolve any package or venv failures	tr-status runs and reports an empty source directory
2	Enlarge swap to 8 GB	free -h shows 8.0Gi
3	Run tr-model; confirm the Hugging Face tag resolves	ollama list shows gams3:q8
4	Generate fixtures; run the full pipeline over them	Translated fixtures appear; tr-lint produces a report
5	Measure real generation and prefill rates	Two tok/s figures recorded from --verbose
6	Test whether Qwen vision input works on a page image	Either a transcription, or a definite failure documented
7	Tune the abbreviation list against fixture output	No segment splits at št. čl. odst. d.o.o.
8	Verify disk encryption; report the finding	lsblk output interpreted, decision recorded

#	Task	Done when
9	Confirm no cloud-routed Ollama models	ollama list shows no -cloud tags
10	Set up systemd-inhibit and tmux for long runs	A fixture batch survives a lid close

Task 5 replaces every throughput estimate in the earlier documents with measurement. Task 6 is the highest-value experiment: if vision input works, the dual-engine OCR cross-check becomes available and the largest risk in the project gets materially smaller.

4.1 Testing without waiting on inference

Iterating on the pipeline against a 13 GB model at four tokens per second is slow and unnecessary. A mock server that mimics the Ollama API makes the whole pipeline testable in seconds, and lets you inject deliberate faults to confirm the linter catches them. Run the fixture corpus through it after any change to segmentation, the non-translatable patterns or the workers: the whole set completes in seconds and tr-lint reports the injected NUM fault. It does not tell you anything about the model — that is task 5.

```
cat > /tmp/mock_ollama.py <<'EOF'
import json, http.server
class H(http.server.BaseHTTPRequestHandler):
    def log_message(self,*a): pass
    def do_POST(self):
        n=int(self.headers["Content-Length"])
        d=json.loads(self.rfile.read(n)); p=d["prompt"]
        if "211" in p: out=p.replace("211","") # fault: lost number
        else: out="EN["+p+"]"
        b=json.dumps({"response":out}).encode()
        self.send_response(200)
        self.send_header("Content-Type","application/json")
        self.send_header("Content-Length",str(len(b)))
        self.end_headers(); self.wfile.write(b)
http.server.ThreadingHTTPServer(("127.0.0.1",11499),H).serve_forever()
EOF

(setuid python3 /tmp/mock_ollama.py >/tmp/mock.log 2>&1 &)
export TR_OLLAMA=http://127.0.0.1:11499 TR_MODEL=mock
tr-run && tr-lint # lint should report the injected NUM fault
```

5. What must not be done in a session

These are operator-only, in a plain terminal with no assistant attached.

Activity	Why it stays outside
Batch runs over the real corpus	Progress output and errors quote source text
Running tr-inventory over a real drop	It opens every client file to detect its language
Opening the container (case-open)	Denied by rule; open it only in a plain terminal
OCR verification against page images	Requires reading the documents. Human work regardless
Glossary extraction from real	Use the local Qwen model for the candidate list, not Claude

Activity	Why it stays outside
documents	
Reading logs from a real run	Logs may contain segments
Diagnosing a failure on a real file	Reproduce with a fixture instead

Glossary extraction deserves a note. Drafting candidate terms from the corpus is a good use of a language model, and the already-installed Qwen model is fast, capable, and entirely local. Run it directly through Ollama rather than through any assistant.

```
# operator-only terminal, no assistant attached
cat work/ocr/*.txt \
| ollama run qwen3.6 "List recurring legal terms, institution names, and
procedural phrases in this text. Output one per line, no commentary."
```

6. What to bring back to this chat

Chat remains useful for design and for documents. Bring back anything that is not case content:

- Measured tokens-per-second figures, so the planning arithmetic can be replaced with real numbers.
- The result of the Qwen vision test, which determines whether the dual-engine OCR design is available.
- The unique-string ratio from the real spreadsheet survey — the ratio and counts, not the strings.
- The Phase 1 timing ratio and which error classes cost the translator time. This is the decision.
- Any design question where the trade-off is not obvious.

Do not paste source sentences, OCR output, translated text, glossary contents, or run logs. Counts, ratios, timings, and error class names carry everything needed to make decisions here.

7. Adjustments to the Operating Manual

The manual stands as written. Three amendments follow from this handover.

Section	Amendment
S2	The kit lives under ~/Claude_Stuff/cli_projects/, a separate branch from ~/translation-work/. That disjointness is a confidentiality control, not tidiness
S3.1	Adjust the username in .claude/settings.json deny paths after unpacking
S6	Phase 1 timing runs in a plain terminal. The drafts and the translator’s edits are case material

One addition to the open items: confirm the container is in place before the first real batch. That single choice makes the boundary in Section 2 structural rather than procedural, and procedural controls are the ones that fail quietly at eleven at night in week three.